SSAD/SOFTWARE ENGINEERING

Unit 4: Project Management & Version Control

Rgpvnotes.tech - Your Academic Excellence Partner

UNIT 4: Project Management Concepts | Configuration & Release Management | Git & Version Control | Release Planning | Change Management | Software Maintenance | Project Metrics

Table of Contents

Topic 1: Project Management Concepts

Topic 2: Configuration and Release Management

Topic 3: Version Control and its tools (Git)

Topic 4: Release Planning

Topic 5: Change Management

Topic 6: Software Maintenance

Topic 7: Project Metrics

Revision Notes

Important Questions

1 Project Management Concepts

Definition:

Software Project Management is the art and science of planning, organizing, staffing, monitoring, and controlling software development activities to deliver a quality product within specified constraints of time, cost, and scope.

Detailed Explanation:

Developing software is a complex endeavor that requires more than just writing code. Without proper

management, projects frequently overrun budgets, miss deadlines, or fail to meet client requirements. Project Management acts as the glue that holds the entire Software Development Life Cycle (SDLC) together.

Effective software project management revolves around the **4 P's**:

1. People: The most crucial element. Managing teams, resolving conflicts, assigning roles, and ensuring motivation.

2. Product: Clearly defining the software objectives, scope, and deliverables before development begins.

3. Process: Selecting the right SDLC framework (Agile, Waterfall, etc.) and breaking down the work into manageable tasks.

4. Project: Planning, monitoring, and controlling the execution to avoid risks and delays.

A project manager (PM) must balance the "Iron Triangle" of project management: **Scope, Time, and Cost**. If a client wants to increase the Scope (add more features), the PM must adjust Time (delay the deadline) or Cost (hire more developers). Quality is at the center of this triangle and is affected if the balance is broken.

🔍 Key Activities in Project Management:

- ✓ **Project Planning:** Defining objectives, scope, and creating a work breakdown structure (WBS).
- ✓ **Scheduling:** Allocating timeframes using Gantt charts or PERT/CPM charts.
- ✓ **Cost Estimation:** Predicting financial resources required (COCOMO, Function Points).
- ✓ **Risk Management:** Identifying potential risks (e.g., team members leaving, tech failures) and creating mitigation plans.
- ✓ **Resource Allocation:** Assigning hardware, software, and human resources efficiently.

📊 The Iron Triangle of Project Management

```
                    SCOPE
                      /\
                     /  \
                    /    \
                  /QUALITY\
                 /________\
              TIME       COST

(A change in one constraint forces a change in the others)
```

💡 Real-Life Example:

Building a Banking App: A project manager is given a budget of $100,000 and 6 months to build a banking app. During development, the client requests a new "Crypto Transfer" feature (Scope Increase). The PM evaluates and states that adding this will require an extra 1 month (Time Increase) and $15,000 (Cost Increase). This is active project management.

🌍 Applications:

→ Handling large-scale enterprise software (e.g., ERP systems).

→ Managing remote, distributed software development teams.

→ Ensuring compliance in critical systems like healthcare or aviation software.

Hinglish Explanation:

Project Management ka matlab hai software project ko time par, diye gaye budget mein, aur sahi features ke sath complete karna. Agar client extra features maangta hai (Scope), toh Time aur Cost dono badhane padte hain. Isme planning, scheduling aur risk management sabse important hote hain.

2 Configuration and Release Management

Definition:

Software Configuration Management (SCM) is the discipline of identifying, organizing, and controlling changes to the software throughout its lifecycle. Release Management is the process of planning, building, testing, and deploying these configured software versions to the end-users.

Detailed Explanation:

Software development is highly dynamic. Multiple developers write code, clients change requirements, and environments update. Without **Software Configuration Management (SCM)**, a project would descend into chaos—developers would overwrite each other's code, and it would be impossible to know which version of the software is currently running in production.

SCM identifies "Configuration Items" (SCIs). An SCI is any document, piece of code, or data structure that is part of the software process. SCM establishes a **Baseline**, which is a formally approved version of an SCI. Once a baseline is set, changes can only be made through formal Change Control procedures.

Release Management takes over when the code is ready. It packages the software, scripts, configuration files, and documentation into a single deployable unit. It ensures that the transition from the developer's laptop to the testing environment, and finally to the live production server, happens smoothly without breaking the system.

Core Activities of SCM:

✓ **Identification:** Labeling software items and their versions (e.g., LoginModule v1.2).

✓ **Version Control:** Managing different versions of the software concurrently.

✓ **Change Control:** Evaluating and approving/rejecting changes before they are applied.

✓ **Configuration Status Accounting:** Recording and reporting the status of items and change requests (e.g., "What changed in v2.0?").

✓ **Configuration Auditing:** Verifying that the software matches its baseline and requirements before

release.

📊 Configuration Management Flow

```
[Developer Changes] ──► [Change Request]
                                  │
                                  ▼
[Configuration Audit] ◄── [Change Control Board (CCB)]
        │                   (Approves/Rejects)
                                  ▼
[Update Baseline] ──► [Release Management] ──► [Production]
```

✅ Advantages:

✓ Prevents code overwriting by multiple developers.

✓ Provides an exact history of what changed, who changed it, and why.

✓ Enables easy rollback to a previous stable version if a new bug appears.

❌ Disadvantages:

✗ Requires strict discipline and administrative overhead.

✗ Learning curve for SCM tools can be steep for new developers.

👉 ***Hinglish Explanation:***
Configuration Management (SCM) ka kaam hai software mein hone wale har change ka record rakhna, taaki koi file galti se delete ya overwrite na ho. Release Management us final ban chuke software ko smoothly user ke system ya server par deploy karne ka process hai.

3 Version Control and its tools (Git)

📌 Definition:

Version Control Systems (VCS) are software tools that help teams manage changes to source code over time. **Git** is the most popular Distributed Version Control System (DVCS) that allows multiple developers to work collaboratively, tracking every modification in a secure and organized manner.

📖 Detailed Explanation:

Before Version Control, developers used to save files like "project_final.zip", "project_final_v2.zip", which was unmanageable. VCS solves this by keeping a timeline of every modification. If a mistake is made, developers can turn back the clock and compare earlier versions of the code to help fix the mistake while

minimizing disruption.

There are main types of VCS:

1. Centralized VCS (e.g., SVN): There is one central server. Developers checkout files, edit, and commit back. If the server goes down, nobody can collaborate.

2. Distributed VCS (e.g., Git): Every developer has a complete local copy of the entire repository history on their machine. It is fast, allows offline work, and has no single point of failure.

Git uses a unique three-tree architecture on a local machine:

- **Working Directory:** Where you actually write and edit files.
- **Staging Area (Index):** A pre-commit holding area where you group changes you want to save together.
- **Local Repository (.git):** Where Git permanently stores the committed snapshots.

Finally, code is pushed to a **Remote Repository** (like GitHub, GitLab, or Bitbucket) to share with the team.

🔧 Crucial Git Concepts & Commands:

- ✓ **Commit (`git commit`):** Saves a snapshot of the staged changes permanently in the local repo.
- ✓ **Branch (`git branch`):** A parallel version of the repository. Developers create branches (e.g., 'feature-login') to work without affecting the main code.
- ✓ **Merge (`git merge`):** Combining changes from one branch into another (e.g., merging a feature into the main branch).
- ✓ **Clone (`git clone`):** Downloading a copy of a remote repository to your local machine.
- ✓ **Push/Pull (`git push / git pull`):** Uploading local commits to the remote server / Downloading updates from the remote server.

📊 Git Workflow Architecture

```
[Working Directory] ──(git add)──▶ [Staging Area]
                                          |
                                   (git commit)
                                          |
                                          ▼
[Remote Repo] ◀──(git push)── [Local Repository]
(GitHub)      ──(git pull)──▶
```

💡 Real-Life Example:

Two developers, Rahul and Priya, are building a website. Rahul creates a branch to build the "Payment Gateway", while Priya creates a branch for the "UI Redesign". Both work simultaneously on their laptops using Git. Once finished, they **commit** their code and **push** it to GitHub. The project lead reviews their code and **merges** both branches into the main website without any code colliding.

🌍 Applications:

→ Tracking code history in all modern software projects.

→ Collaborating on Open Source software globally.

→ Automating deployments (CI/CD pipelines trigger when code is pushed to Git).

Hinglish Explanation:

Git ek tool hai jo code ki har change ka backup rakhta hai. Isse multiple log ek hi project par bina ek-dusre ka code delete kiye kaam kar sakte hain. Branching ke through aap naya feature safe environment mein bana sakte hain aur baad mein 'merge' kar sakte hain.

4 Release Planning

Definition:

Release Planning is the process of deciding which features, bug fixes, and enhancements will be delivered to the end-users in upcoming versions of the software, and defining the schedule and resources required for that delivery.

Detailed Explanation:

You cannot build and deliver a massive software project all at once. Release planning breaks the project down into strategic, incremental releases. It aligns the development team's capacity with the business goals of the client or company. It answers the questions: "*What are we delivering?*" and "*When are we delivering it?*".

In traditional (Waterfall) methods, a release might happen once a year, containing massive updates. In Agile methodologies, release planning is done frequently (e.g., every 1-3 months), consisting of multiple short "Sprints" or iterations. Agile release planning is highly flexible, allowing the business to shift priorities based on market feedback.

A successful release plan considers:

1. **Velocity/Capacity:** How much work the team can realistically complete.
2. **Prioritization:** Deciding which features yield the highest business value.
3. **Dependencies:** Recognizing that Feature B cannot be built until Feature A is released.

Steps in Release Planning:

- ✓ **Vision & Roadmap:** Understanding the long-term goal of the product.
- ✓ **Backlog Grooming:** Reviewing and prioritizing the list of required features (Product Backlog).
- ✓ **Estimation:** Assigning effort/time estimates to features (e.g., Story Points).
- ✓ **Sprint Mapping:** Allocating prioritized features across multiple upcoming sprints/iterations.
- ✓ **Release Schedule:** Setting a target date to deploy the accumulated iterations to the live environment.

📊 Agile Release Planning Hierarchy

```
PRODUCT VISION
      │
      ▼
PRODUCT ROADMAP
      │
      ▼
RELEASE PLAN (e.g., Release 1.0: Q1 2026)
    ├── Sprint 1 (Weeks 1-2)
    ├── Sprint 2 (Weeks 3-4)
    └── Sprint 3 (Weeks 5-6) ──► [DEPLOYMENT]
```

✅ Advantages:

✓ Sets clear expectations for clients and stakeholders.

✓ Allows marketing and sales teams to plan campaigns around feature launches.

✓ Provides a clear target for the development team.

❌ Disadvantages:

✗ Over-promising in a release plan can lead to developer burnout.

✗ Strict release dates can sometimes compromise code quality.

👉 ***Hinglish Explanation:***
Release planning ka matlab hai ek schedule banana ki software ke kaunse features kis mahine ya date ko public ke liye launch kiye jayenge. Isme team ki speed aur features ki priority dekhi jati hai, taaki users ko time par updates milte rahein.

5 Change Management

📌 Definition:

Change Management in software engineering is a systematic, controlled process for proposing, evaluating, approving, and implementing changes to a software system after its baseline requirements have been established.

📖 Detailed Explanation:

Change is inevitable in software development. Business rules change, new laws are passed, or clients simply change their minds. However, uncontrolled changes lead to "Scope Creep," where the project grows endlessly, deadlines are missed, and budgets explode. Change Management provides a formal gateway to handle these modifications safely.

The core of Change Management is the **Change Control Board (CCB)**. The CCB is a group of stakeholders (Project Manager, Lead Architect, Client Representative) who evaluate every requested change. They do an **Impact Analysis** to determine how the proposed change will affect the existing architecture, the timeline, and the budget. If the change is too risky or expensive, the CCB rejects it. If approved, the project plan is formally updated.

🔍 The Change Management Process:

1. ✓ **Change Request (CR):** A formal document is submitted detailing the requested change and why it is needed.
2. ✓ **Impact Analysis:** Technical and management teams analyze how much time, money, and effort the change will cost.
3. ✓ **CCB Evaluation:** The Change Control Board reviews the analysis and votes to approve, reject, or defer the change.
4. ✓ **Implementation:** If approved, developers make the change under Version Control.
5. ✓ **Verification & Closure:** Quality Assurance tests the change to ensure it works and hasn't broken existing features. The CR is then closed.

💡 Real-Life Example:

During the development of a food delivery app, the government introduces a new "Digital Tax" rule. The client submits a **Change Request** to update the checkout calculation logic. The CCB performs an Impact Analysis, realizes it will take 1 week and $2000 to implement, and approves it because it is legally mandatory. The timeline is formally extended by 1 week.

🌍 Applications:

→ Preventing scope creep in fixed-budget freelance contracts.

→ Managing updates in enterprise software safely.

→ Ensuring ISO or CMMI compliance in corporate IT environments.

👉 ***Hinglish Explanation:***
Jab project chal raha ho aur client naya feature maange, toh seedha code change nahi karte. Ek formal request banti hai, uski cost aur time check hota hai (Impact Analysis), aur ek committee (CCB) use approve karti hai. Is discipline ko Change Management kehte hain, isse project out-of-control nahi hota.

6 Software Maintenance

Definition:

Software Maintenance is the process of modifying a software product after it has been delivered to the customer. The goal is to correct faults, improve performance, or adapt the software to a changed operating environment.

Detailed Explanation:

Many people assume development is the most expensive phase of software. In reality, **Software Maintenance consumes over 60% of the total lifecycle costs**. Software does not "wear out" physically like a car engine, but it does "deteriorate" logically over time as hardware changes, operating systems update, and user needs evolve. If software is not maintained, it becomes obsolete.

Maintenance is categorized into four distinct types. Understanding these types is crucial because they require different approaches and budgeting:

1. Corrective Maintenance (Bug Fixing):

Reactive maintenance. It involves fixing errors and bugs that users discover after the software is deployed. (e.g., Fixing a crash that happens when a user uploads a large image).

2. Adaptive Maintenance (Environment Updates):

Modifying the software to keep it usable in a changing environment. (e.g., Updating an iOS app so it works on the newly released iOS 18 version, or moving from a local database to a cloud database).

3. Perfective Maintenance (New Features & Enhancements):

Improving the software by adding new capabilities or enhancing existing ones based on user feedback. This makes up the majority of maintenance work. (e.g., Adding a "Dark Mode" to an application).

4. Preventive Maintenance (Refactoring):

Modifying the software to prevent future problems. This involves code restructuring, optimization, and updating documentation without changing the user-facing functionality. (e.g., Rewriting a slow database query to be faster before user traffic scales up).

Key Challenges in Maintenance:

- ✓ **Poor Documentation:** Original developers leave, and new maintainers struggle to understand the undocumented code.
- ✓ **Spaghetti Code:** Rapid, messy patches degrade the architecture, making future changes harder.
- ✓ **Legacy Systems:** Maintaining outdated technologies (e.g., COBOL systems in banks) is expensive and hard to staff.
- ✓ **Regression Bugs:** Fixing one bug accidentally breaks another feature.

Distribution of Maintenance Effort

```
[ Perfective Maintenance ] : ~50% (Adding new features)
[ Adaptive Maintenance   ] : ~25% (Adapting to OS/Hardware)
[ Corrective Maintenance ] : ~20% (Fixing bugs)
[ Preventive Maintenance ] : ~5%  (Refactoring code)
```

✅ Advantages of good maintenance:

✓ Extends the lifespan and ROI of the software.

✓ Keeps the product secure against new cyber threats.

✓ Ensures high user satisfaction by delivering modern features.

Hinglish Explanation:
Software banne ke baad usme bugs theek karna (Corrective), naye OS ke hisaab se update karna (Adaptive), naye features daalna (Perfective) aur code ko future-proof banana (Preventive) Maintenance kehlata hai. Total budget ka sabse bada hissa maintenance mein hi kharch hota hai.

7 Project Metrics

Definition:

Software Project Metrics are quantitative, measurable values used by project managers to estimate, track, and evaluate the progress, quality, and productivity of a software development process and product.

Detailed Explanation:

Lord Kelvin famously said, "*If you cannot measure it, you cannot improve it.*" In software engineering, relying on gut feelings ("I think we are 80% done") leads to disaster. Project Metrics provide objective, mathematical data to understand the exact health of a project.
Metrics help Project Managers in multiple ways:
- **Estimation:** Using past metrics to predict how much time/money a new project will take.
- **Quality Control:** Tracking the number of bugs found per week to evaluate software stability.
- **Productivity:** Understanding how much output the team is generating.

Types of Metrics:

- **Process Metrics:** Measure the efficiency of the software development process itself (e.g., Average time taken to fix a bug).
- **Project Metrics:** Track the status of the current project against its plan (e.g., Cost Variance, Schedule Variance).
- **Product Metrics:** Measure the characteristics of the software being built (e.g., Code complexity, Size of the software).

Important Measurement Approaches:

1. Size-Oriented Metrics (LOC):
Measures the software based on the Lines of Code (LOC) written. Metrics are calculated as *Errors per 1000 LOC (KLOC)* or *Cost per KLOC*. While easy to count, it is flawed because different programming languages require different amounts of code for the same task.
2. Function-Oriented Metrics (Function Points - FP):

Measures functionality delivered to the user, regardless of the programming language. It counts external inputs, outputs, user inquiries, internal files, and external interfaces. FP is a much more accurate measure of true software value.

Common Project Metrics:

- ✓ **Defect Density:** Number of defects / Size of software (KLOC or FP). Shows quality.
- ✓ **Schedule Variance (SV):** Expected Time vs Actual Time taken. Shows if project is delayed.
- ✓ **Cost Variance (CV):** Expected Budget vs Actual Budget spent. Shows financial health.
- ✓ **Productivity:** FP produced / Person-Months of effort. Shows team efficiency.
- ✓ **MTBF (Mean Time Between Failures):** Average time the software runs without crashing.

Calculation Example:

A team finishes a project in 5 Person-Months. The software is 10,000 Lines of Code (10 KLOC). During testing, 25 errors were found.
- **Productivity:** 10 KLOC / 5 months = 2 KLOC per month.
- **Defect Density:** 25 errors / 10 KLOC = 2.5 errors per KLOC.

Hinglish Explanation:
Metrics ka matlab hai project ko numbers aur data mein naapna. Project kaisa chal raha hai, kitne bugs aa rahe hain, budget cross toh nahi ho raha? LOC (Lines of Code) purana tareeka hai size naapne ka, aaj kal Function Points (FP) use hota hai jo zyada accurate hota hai.

QUICK REVISION NOTES

1. Project Management:
Managing the 4 P's (People, Product, Process, Project) to deliver software balancing the Iron Triangle constraints: Scope, Time, Cost, and Quality.

2. Configuration & Release Mgmt:
SCM tracks all changes using version control, creates baselines, and prevents code chaos. Release management deploys the stable code to production.

3. Git & Version Control:
Git is a Distributed VCS. Core commands: Add (stage), Commit (save locally), Push (upload to remote), Pull (download), Branch (parallel work), Merge (combine).

4. Release Planning:
Mapping out which features will be delivered in which upcoming timeline/sprint. Prioritizes business value against development team capacity.

5. Change Management:
Handling scope changes formally. Change Request (CR) ➔ Impact Analysis ➔ Change Control Board (CCB) Approval ➔ Implementation ➔ Testing.

6. Software Maintenance:
Modifying software post-delivery. Corrective (bugs), Adaptive (new OS), Perfective (new features - takes ~50% effort), Preventive (refactoring/optimization).

7. Project Metrics:
Quantitative measurements. Size-oriented (LOC - Lines of Code) vs Function-oriented (FP - Function points). Key metrics: Defect density, Productivity, Schedule Variance.

📝 IMPORTANT QUESTIONS (6 Marks Each)

Q1. Explain the role of a Software Project Manager. Discuss the 'Iron Triangle' and the 4 P's of project management. 6 Marks

Expected Answer Points:

- Definition of SPM (planning, organizing, controlling).
- Role of PM: risk management, scheduling, resource allocation, communication.
- 4 P's detail: People (team), Product (scope/objectives), Process (framework), Project (execution).
- Iron Triangle diagram: Scope, Time, Cost interacting with Quality.
- Explain trade-offs: e.g., if scope increases, time or cost must increase.

Q2. What is Software Configuration Management (SCM)? Why is it necessary in a collaborative development environment? Explain its core activities. 6 Marks

Expected Answer Points:

- Define SCM (identifying and controlling changes).
- Necessity: prevents code overwrite, provides history, handles concurrent development, allows rollbacks.
- Configuration Items (SCIs) and Baselines.
- Core Activities: Identification, Version Control, Change Control, Configuration Status Accounting, Configuration Auditing.

Q3. Differentiate between Centralized and Distributed Version Control Systems. Explain the basic architecture and workflow of Git. 6 Marks

Expected Answer Points:

- Centralized (SVN): Single server, single point of failure, requires network.
- Distributed (Git): Every dev has full local copy, offline work possible, highly resilient.
- Git Architecture: Working Directory ➔ Staging Area ➔ Local Repo ➔ Remote Repo.
- Commands explanation: git add, git commit, git push, git pull, git branch, git merge.

Q4. Define Software Maintenance. Elaborate on the four different types of software maintenance with appropriate real-world examples. 6 Marks

Expected Answer Points:

- Definition: Modifying software post-delivery to fix, adapt, or improve.
- Corrective: Fixing bugs/crashes (Example: fixing a broken checkout button).
- Adaptive: Modifying for environment changes (Example: updating app for new Android version).
- Perfective: Adding new features/enhancements (Example: adding dark mode). Highest effort (~50%).
- Preventive: Code refactoring to prevent future issues (Example: optimizing a slow database query).

Q5. What are Software Metrics? Differentiate between Size-Oriented (LOC) and Function-Oriented (FP) metrics. Why are metrics crucial for project management? 6 Marks

Expected Answer Points:

- Define metrics (quantitative measures of process, project, product).
- Importance: Estimation, tracking progress, measuring quality, evaluating productivity.
- Size-Oriented (LOC): Measures physical lines of code. Flawed because it varies wildly by programming language.
- Function-Oriented (FP): Measures business functionality delivered to the user (inputs, outputs, interfaces). Language independent.
- Examples of derived metrics: Defect density (defects/KLOC), Productivity (FP/person-month).

These notes are comprehensive and exam-ready

Study thoroughly, practice the Git concepts, and ace your examination!

SSAD/Software Engineering - Unit 4 Notes

Source: Rgpvnotes.tech | Premium Academic Content for Students

© 2026 - Your Academic Excellence Partner

These notes are created for educational purposes to help students understand complex concepts in a structured manner.